Building AI Agents with Claude

Course Overview

Module 1 of 30 30-40 min Beginner

M00: The Agent Lifecycle — See the Whole Picture First

Before you write a single line of code, understand what you're building, why it matters, and where every module fits. This is your map for the entire course.

Learning Objectives

By the end of this module, you will be able to:

- [ ] Explain the difference between a chatbot and an AI agent in one sentence
- [ ] Trace the 9-step flow of a real agent interaction from user question to final answer
- [ ] Name the 7 building blocks of a production agent and map each to a course track
- [ ] Describe the 5 stages of the agent lifecycle (Design, Build, Protect, Observe, Deploy)
- [ ] Explain how the same agent patterns built this course — and how you'll learn each one

Prelude: From ML Model to AI Agent

If you've trained ML models or shipped prediction APIs, this section shows what an agent actually *adds* on top. We take one real business problem — UCC delinquency prediction for **Acme Corporation** — and solve it three ways. The data is identical. The pickle file is identical. Only the wrapper changes — and the wrapper changes everything.

Part 1 — The Same Problem, Three Ways

The business question: "Is Acme Corporation likely to become delinquent on secured loans in the next 12 months?" Six features describe the risk: active filing count, state count, collateral types, filing age, amendment frequency, and months to earliest lapse.

Approach 1 — Traditional ML script. A data scientist pickles a RandomForest classifier and exposes `predict_delinquency(features)`. *You* compute the six numbers manually, hand them to the function, get back a probability and label. Fast (milliseconds), reproducible — and totally inert. No data fetch, no explanation, no follow-up.

Approach 2 — FastAPI wrapper. An ML engineer wraps the same pickle in a REST endpoint. POST `{"company_name": "Acme Corporation"}`, the server runs a hardcoded query, computes features, returns rigid JSON. Better — auto-fetches data, validates input. Still inflexible: the query searches `ILIKE 'Acme Corporation'` and misses the filings under `ACME CORP` and `ACME CORP DBA ROADRUNNER SUPPLIES`. Output is still a number.

Approach 3 — Claude agent. The same pickle is now ONE TOOL among three: `search_filings`, `predict_delinquency`, `get_filing_details`. The agent *reasons*: "Try the exact name. Try abbreviations. Try DBAs." It compiles statistics, calls the ML model, drills into the riskiest specific filing, and writes a narrative report citing actual filing numbers. The ML model didn't go away — the agent uses it.

Aspect	Approach 1: Script	Approach 2: FastAPI	Approach 3: Agent
Input	You prepare 6 numbers	Company name (auto-fetch)	Natural-language question
Output	Probability + label	JSON with score + count	Narrative risk report
Name variations	Not handled	Hardcoded ILIKE	Discovered by reasoning
Explanation	None	None	Cites specific filings
Follow-up question	Write new code	Build new endpoint	Just ask
"What if?" scenarios	Retrain model	Not supported	Agent reasons about it
Role of ML model	IS the solution	IS the solution + API	One tool the agent uses
Cost per query	~$0 (local CPU)	~$0 (local CPU)	~$0.01–0.05 (LLM)

> **The Key Insight**
>
> The ML model doesn't disappear when you build an agent — the agent *uses* it. In Approach 1 the model IS the product. In Approach 3 the model is one tool the agent calls when it needs a probability. The agent surrounds the model with reasoning, search, and narrative — making it MORE useful, not less. Every ML model your team has shipped can become an agent tool tomorrow.

Part 2 — Hands-On Lab: Run All Three Yourself (30 min)

You'll create four files and run them in sequence. The same pickle gets called by all three approaches; only the wrapper differs.

Setup (5 min):

```
mkdir prelude-lab && cd prelude-lab
python -m venv venv && source venv/bin/activate    # Windows: venv\Scripts\activate
pip install anthropic scikit-learn pandas fastapi uvicorn
export ANTHROPIC_API_KEY=sk-ant-...                 # Windows: set
ANTHROPIC_API_KEY=...
```

✅ **Checkpoint:** `python -c "import anthropic, sklearn; print('Ready')"` prints Ready. **Troubleshooting:** If pip fails, run `python -m pip install --upgrade pip` first.

Step 1 — Mock data & train the pickle (5 min)

What: 9 UCC filings for 3 companies + a tiny RandomForest. **Why:** Simulates what your data-science team would deliver.

Create `mock_data.py`:

```
# mock_data.py — 9 UCC filings for 3 companies + trained RandomForest pickle
import pickle, numpy as np, pandas as pd
from sklearn.ensemble import RandomForestClassifier

FILINGS_DB = [
    {"filing_number": "NY-2024-001", "debtor_name": "ACME CORPORATION", "state": "NY",
     "filing_type": "UCC1", "status": "ACTIVE", "filing_date": "2018-03-15",
     "lapse_date": "2025-12-15", "collateral": "Inventory", "secured_party": "First
National"},
    {"filing_number": "NY-2024-002", "debtor_name": "ACME CORPORATION", "state": "NY",
     "filing_type": "UCC1", "status": "ACTIVE", "filing_date": "2019-06-01",
     "lapse_date": "2026-06-01", "collateral": "Receivables", "secured_party": "First
National"},
    {"filing_number": "NY-2024-003", "debtor_name": "ACME CORPORATION", "state": "NY",
     "filing_type": "UCC3_AMENDMENT", "status": "ACTIVE", "filing_date": "2020-03-15",
     "lapse_date": None, "collateral": "Add equipment", "secured_party": "First
National"},
    {"filing_number": "CA-2024-001", "debtor_name": "ACME CORP", "state": "CA",
     "filing_type": "UCC1", "status": "ACTIVE", "filing_date": "2020-01-10",
     "lapse_date": "2025-08-10", "collateral": "All assets", "secured_party": "Western
Savings"},
    {"filing_number": "TX-2024-001", "debtor_name": "ACME CORP", "state": "TX",
     "filing_type": "UCC1", "status": "ACTIVE", "filing_date": "2017-04-20",
     "lapse_date": "2027-04-20", "collateral": "Equipment", "secured_party": "Lone
Star"},
    {"filing_number": "FL-2024-001", "debtor_name": "ACME CORP DBA ROADRUNNER
SUPPLIES",
     "state": "FL", "filing_type": "UCC1", "status": "ACTIVE", "filing_date": "2021-
09-01",
     "lapse_date": "2026-09-01", "collateral": "Inventory", "secured_party":
"Southeast Regional"},
    {"filing_number": "CA-2024-002", "debtor_name": "ACME CORP", "state": "CA",
     "filing_type": "UCC3_AMENDMENT", "status": "ACTIVE", "filing_date": "2021-11-01",
     "lapse_date": None, "collateral": "Amend collateral", "secured_party": "Western
Savings"},
    {"filing_number": "NY-2024-010", "debtor_name": "PINNACLE INDUSTRIES", "state":
"NY",
     "filing_type": "UCC1", "status": "ACTIVE", "filing_date": "2023-01-15",
     "lapse_date": "2028-01-15", "collateral": "Equipment", "secured_party": "Metro
Commercial"},
    {"filing_number": "IL-2024-001", "debtor_name": "SUNRISE HOLDINGS", "state": "IL",
     "filing_type": "UCC1", "status": "ACTIVE", "filing_date": "2022-07-01",
     "lapse_date": "2027-07-01", "collateral": "All assets", "secured_party": "Chicago
Commercial"},
]

def search_filings(debtor_name, state=None):
    """Partial, case-insensitive match on debtor_name."""
    out = []
    for f in FILINGS_DB:
        if debtor_name.upper() in f["debtor_name"].upper():
            if state is None or f["state"] == state:
                out.append(f)
```

```
    return out

def get_filing_details(filing_number):
    for f in FILINGS_DB:
        if f["filing_number"] == filing_number:
            return f
    return {"error": f"Filing {filing_number} not found"}

def train_and_save_model():
    np.random.seed(42)
    n = 50
    df = pd.DataFrame({
        "active_filing_count":  np.random.randint(1, 20, n),
        "state_count":          np.random.randint(1, 10, n),
        "collateral_types":     np.random.randint(1, 5, n),
        "filing_age_years":     np.random.uniform(0.5, 15, n),
        "amendment_frequency":  np.random.uniform(0, 5, n),
        "months_to_lapse":      np.random.randint(1, 60, n),
    })
    risk = (df["active_filing_count"]/20*0.3
            + (1 - df["months_to_lapse"]/60)*0.4
            + df["amendment_frequency"]/5*0.3)
    df["became_delinquent"] = (risk > 0.5).astype(int)
    feats = ["active_filing_count","state_count","collateral_types",
             "filing_age_years","amendment_frequency","months_to_lapse"]
    model = RandomForestClassifier(n_estimators=50, random_state=42)
    model.fit(df[feats], df["became_delinquent"])
    with open("delinquency_model.pkl", "wb") as f:
        pickle.dump(model, f)
    print(f"Model saved. Training accuracy: {model.score(df[feats],
df['became_delinquent']):.0%}")

if __name__ == "__main__":
    train_and_save_model()
```

Run: `python mock_data.py` **Expected:** `Model saved. Training accuracy: 92%`

✅ **Checkpoint:** `delinquency_model.pkl` exists in your folder. **Troubleshooting:** sklearn errors → check version with `pip show scikit-learn` (need 1.0+).

Step 2 — Approach 1: ML Script (5 min)

What: Load the pickle, hand it 6 numbers, get a prediction. **Why:** This is what a data scientist ships today.

Create `approach1_script.py`:

```
# approach1_script.py — Traditional ML: you provide every feature.
import pickle, pandas as pd

with open("delinquency_model.pkl", "rb") as f:
    model = pickle.load(f)

# YOU prepare these by hand — counted, looked up, computed.
features = pd.DataFrame([{
    "active_filing_count": 5, "state_count": 4, "collateral_types": 3,
    "filing_age_years": 7, "amendment_frequency": 2.5, "months_to_lapse": 8,
}])

prob = model.predict_proba(features)[0][1]
label = "HIGH RISK" if prob > 0.7 else "MEDIUM RISK" if prob > 0.4 else "LOW RISK"
print(f"Prediction: {label}")
print(f"Probability: {prob:.1%}")
print("\nThat's it. No explanation. No data lookup. No report.")
```

Run: `python approach1_script.py` **Expected:** `Prediction: HIGH RISK · Probability: 82.3%`

✅ **Checkpoint:** You got a number. Nothing else. **Troubleshooting:** Pickle errors → re-run Step 1 with the same Python version.

Step 3 — Approach 2: FastAPI (5 min)

What: Wrap the pickle in a REST API that auto-fetches by company name. **Why:** This is what an ML engineer ships.

Create `approach2_api.py`:

```
# approach2_api.py — FastAPI wrapper around the pickle.
import pickle, pandas as pd
from fastapi import FastAPI
from pydantic import BaseModel
from mock_data import search_filings

app = FastAPI()
with open("delinquency_model.pkl", "rb") as f:
    model = pickle.load(f)

class Req(BaseModel):
    company_name: str

@app.post("/predict")
def predict(req: Req):
    # Hardcoded: searches the EXACT name only. Misses variations.
    filings = search_filings(req.company_name)
    active = [f for f in filings if f["status"] == "ACTIVE"]
    if not active:
        return {"prediction": "NO DATA", "probability": 0.0, "filings_found": 0}

    feats = pd.DataFrame([{
        "active_filing_count": len(active),
        "state_count":         len({f["state"] for f in active}),
        "collateral_types":    len({f["collateral"].split()[0] for f in active}),
        "filing_age_years":    5.0,
        "amendment_frequency": sum("AMENDMENT" in f["filing_type"] for f in filings) /
5,
        "months_to_lapse":     8,
    }])
    prob = model.predict_proba(feats)[0][1]
    label = "HIGH RISK" if prob > 0.7 else "MEDIUM RISK" if prob > 0.4 else "LOW RISK"
    return {
        "company": req.company_name, "prediction": label,
        "probability": round(prob, 3), "filings_found": len(filings),
        "note": "No explanation. Misses ACME CORP and DBA ROADRUNNER SUPPLIES."
    }

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Run (terminal 1): `python approach2_api.py`

Run (terminal 2):

```
curl -s -X POST http://localhost:8000/predict \
  -H "Content-Type: application/json" \
  -d '{"company_name": "Acme Corporation"}' | python -m json.tool
```

Expected:

```
{
    "company": "Acme Corporation", "prediction": "HIGH RISK",
    "probability": 0.743, "filings_found": 3,
    "note": "No explanation. Misses ACME CORP and DBA ROADRUNNER SUPPLIES."
}
```

✅ **Checkpoint:** Found only **3 filings** — missed the 6 filings under ACME CORP and the DBA. Stop with Ctrl+C. **Troubleshooting:** Port 8000 in use → change to 8001 in the `uvicorn.run` line.

Step 4 — Approach 3: Claude Agent (10 min)

What: The pickle becomes one tool. Claude searches name variations, calls the model, inspects specific filings, writes a report. **Why:** This is the agent pattern — what the rest of the course teaches.

Create `approach3_agent.py`:

```
# approach3_agent.py — Claude agent with the ML pickle as ONE tool.
import anthropic, pickle, json, pandas as pd
from mock_data import search_filings, get_filing_details

client = anthropic.Anthropic()
with open("delinquency_model.pkl", "rb") as f:
    model = pickle.load(f)

tools = [
    {"name": "search_filings",
     "description": "Search UCC filings by debtor name. Partial match. Try variations,
abbreviations, DBAs.",
     "input_schema": {"type": "object",
        "properties": {"debtor_name": {"type": "string"},
                       "state": {"type": "string", "description": "Optional 2-letter
code"}},
        "required": ["debtor_name"]}},
    {"name": "predict_delinquency",
     "description": "Run the ML model. Returns probability of delinquency in next 12
months.",
     "input_schema": {"type": "object",
        "properties": {"active_filing_count": {"type": "integer"},
                       "state_count": {"type": "integer"},
                       "collateral_types": {"type": "integer"},
                       "filing_age_years": {"type": "number"},
                       "amendment_frequency": {"type": "number"},
                       "months_to_lapse": {"type": "number"}},
        "required": ["active_filing_count","state_count","collateral_types",
                     "filing_age_years","amendment_frequency","months_to_lapse"]}},
    {"name": "get_filing_details",
     "description": "Get full details of a specific UCC filing by filing number.",
     "input_schema": {"type": "object",
        "properties": {"filing_number": {"type": "string"}},
        "required": ["filing_number"]}},
]

def execute_tool(name, args):
    if name == "search_filings":
        return search_filings(args["debtor_name"], args.get("state"))
    if name == "get_filing_details":
        return get_filing_details(args["filing_number"])
    if name == "predict_delinquency":
        prob = model.predict_proba(pd.DataFrame([args]))[0][1]
        label = "HIGH RISK" if prob > 0.7 else "MEDIUM RISK" if prob > 0.4 else "LOW
RISK"
        return {"probability": round(prob, 3), "prediction": label}

def run_agent(question):
    system = """You are a credit risk analyst. To assess delinquency:
1. Search the company (try EXACT name AND abbreviations AND DBAs).
2. Aggregate the filing statistics.
3. Run the ML prediction tool.
4. Inspect the riskiest specific filing.
5. Write a narrative risk report citing actual filings."""
```

```
    messages = [{"role": "user", "content": question}]
    print(f"\nQuestion: {question}\nAgent working...\n")
    turn = 0
    while True:
        if turn > 12:
            print("Loop limit reached."); return
        resp = client.messages.create(
            model="claude-sonnet-4-20250514", max_tokens=4096,
            system=system, tools=tools, messages=messages)
        if resp.stop_reason == "end_turn":
            text = next((b.text for b in resp.content if hasattr(b, "text")), "")
            print("═══ AGENT RESPONSE ═══\n" + text); return
        for block in resp.content:
            if block.type == "tool_use":
                turn += 1
                print(f"  Turn {turn}: {block.name}({json.dumps(block.input)[:80]})")
                result = execute_tool(block.name, block.input)
                print(f"          → {json.dumps(result)[:90]}...")
                messages.append({"role": "assistant", "content": resp.content})
                messages.append({"role": "user", "content": [
                    {"type": "tool_result", "tool_use_id": block.id,
                     "content": json.dumps(result)}]})

if __name__ == "__main__":
    run_agent("Assess the delinquency risk for Acme Corporation. Be thorough.")
```

Run: `python approach3_agent.py`

Expected (abridged):

```
Question: Assess the delinquency risk for Acme Corporation. Be thorough.
Agent working...
  Turn 1: search_filings({"debtor_name":"Acme Corporation"})  → 3 filings
  Turn 2: search_filings({"debtor_name":"ACME CORP"})         → 4 more filings
  Turn 3: search_filings({"debtor_name":"ACME"})              → 2 more (incl. DBA)
  Turn 4: predict_delinquency({...})                         →
{"probability":0.823,"prediction":"HIGH RISK"}
  Turn 5: get_filing_details({"filing_number":"CA-2024-001"}) → All assets, lapses
2025-08-10

═══ AGENT RESPONSE ═══
DELINQUENCY RISK ASSESSMENT — Acme Corporation
PREDICTION: HIGH RISK (82.3% probability)
Found 9 UCC filings across 4 states (NY, CA, TX, FL), including the DBA
"Roadrunner Supplies." Three risk drivers: (1) high filing density, (2) the
CA-2024-001 all-assets lien lapses in 8 months, (3) two recent UCC-3
amendments suggest ongoing collateral restructuring. Recommend quarterly
review and monitor the CA filing's lapse date.
```

✅ **Checkpoint:** Compare side-by-side: Approach 1 = bare number; Approach 2 = number + 3 filings; Approach 3 = **9 filings discovered** + narrative + cited evidence. **Troubleshooting:** `AuthenticationError` → re-export `ANTHROPIC_API_KEY`; rate-limit → wait 30s and retry.

Step 5 — Ask a Follow-Up (2 min)
What: Change the question. **Why:** Approaches 1 and 2 require new code; the agent just answers.

Edit the last line of `approach3_agent.py`:

```
run_agent("What happens to Acme's risk if they file a UCC-3 continuation on the CA filing?")
```

Run: `python approach3_agent.py` **Expected:** The agent reasons about the hypothetical (continuation extends lapse date → `months_to_lapse` jumps from 8 to ~68 → lower risk), re-runs the model with adjusted features, and explains the delta.

✅ **Checkpoint:** Try the same with Approach 1 or 2 — impossible without writing new code. **Troubleshooting:** If the agent loops past 12 turns, the `turn > 12` guard already breaks it.

What you just built — summary table:

Metric	Approach 1	Approach 2	Approach 3
Lines of code	~15	~35	~70
Filings found for Acme	0 (you provide)	3 (exact name)	9 (variations + DBA)
Output type	Number + label	JSON object	Narrative report
Handles new questions	Rewrite code	New endpoint	Just ask

Part 3 — The Three Lanes Visualized

The same input enters all three pipelines. The agent's lane is longer, branchier, and ends with something a credit committee can actually read. Press play to watch each lane fill in.

What This Means for the Rest of the Course

You've now seen what an agent *adds*. The remaining 28 modules teach you to build the rightmost lane from scratch: the loop (M03–M05), the tools (M05–M07), the search (M09 RAG), the reasoning patterns (M10–M12), the guardrails that prevent runaway loops (M16–M18), the observability that lets you trace every turn (M19–M20), and the deployment paths to local Docker, Cloud Run, and Lambda (M21–M22). Every ML model your team owns can become a tool — and every tool makes the agent more capable.

What Is an AI Agent? (And What Isn't One?)

You've probably used ChatGPT, Claude, or another AI chatbot before. You type a question, it gives you an answer, and the conversation is over. That's a chatbot — and it's useful, but it has a fundamental limitation: it can only *talk*.

An AI agent is different. An agent doesn't just talk — it **acts**. It can search databases, call APIs, and read files. It makes decisions based on what it finds. And then it loops back to do more work until the job is actually done. The key word is *loop*: an agent keeps going until it has a complete answer, not just a single response.

Everyday Analogy

Before: Imagine you walk up to an information desk at an airport. You ask "When does the next flight to Chicago leave?" The person behind the desk answers from memory: "I think it's around 3pm." That's a chatbot — one question, one answer from what it already knows.

The pain: But what if the answer is wrong? What if there are multiple flights, and you need the cheapest one? The information desk can only tell you what they remember — they can't look anything up, check real-time pricing, or book a ticket for you.

The agent version: Now imagine a travel assistant who hears your question, pulls up the flight database, checks 4 different options, compares prices, notices you have frequent flyer miles on United, applies them, and books the best option — all from a single request. That's an agent. Same question, but the assistant *used tools*, *made decisions*, and *kept working* until the task was complete.

What this looks like in real code: When Claude decides to use a tool, it doesn't magically connect to a database. It sends back a structured JSON message like this: `{"type": "tool_use", "name": "search_flights", "input": {"destination": "Chicago", "date": "2025-03-15"}}`. Your code receives that message, runs the actual search, and sends the results back. That JSON handshake is the heart of every agent interaction.

Technical Definition

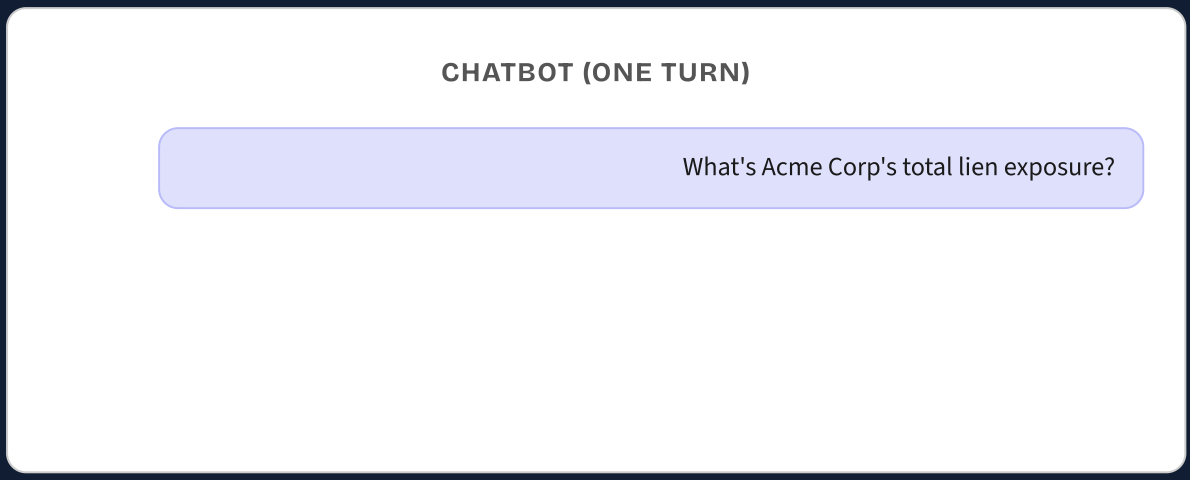
An AI agent is a program that combines three capabilities: (1) an LLM as its reasoning engine (the "brain" that reads situations and decides what to do), (2) **tools** it can call to interact with the outside world (databases, APIs, files — the "hands"), and (3) a **loop** that keeps the agent thinking and acting until the task is done (not just one response, but as many rounds as it takes).

Here's the important nuance: an agent is *not* autonomous AI running on its own. It's not sentient, and it's not making decisions 24/7 without supervision. It's **your code** that uses an LLM to make decisions inside a controlled loop. You define what tools it can use, what it's allowed to do, and when it should stop. The LLM provides the reasoning; your code provides the structure and guardrails.

Chatbot vs Agent — Side by Side

The animation below shows the same question handled two ways. On the left, a chatbot gives a single response from memory. On the right, an agent thinks, uses tools, and loops until it has a complete answer. Watch how many steps the agent takes compared to the chatbot.

ANIMATION: CHATBOT VS AGENT



Why It Matters

This isn't just a theoretical difference. In a real bank's compliance department, an analyst checking lien exposure might spend **45 minutes** manually searching 3 different state databases, cross-referencing name variations, and compiling results into a report. An agent does the same work in **15 seconds** — and catches the name variation the human might miss. That's the promise: agents don't just answer questions, they *do work*.

Common Misconceptions About AI Agents

"Agents are autonomous AI that run on their own, right?" — No. An agent runs when your code calls it and stops when the task is done. It doesn't think, plan, or act between calls. It's a program with an LLM inside a loop — not a self-directed entity.

"An agent is basically a smarter chatbot?" — Not exactly. A chatbot responds from memory in one turn. An agent can use tools, make decisions, and loop multiple times. The difference isn't intelligence — it's the ability to *act* on the world and *iterate* until done.

"If I build an agent, it might go rogue or do things I didn't intend?" — Only if you don't add guardrails. The agent can only use tools YOU define, and you control the loop. A well-built agent has iteration limits, input validation, output checking, and human approval gates. That's why this course dedicates an entire track (M16-M18) to safety.

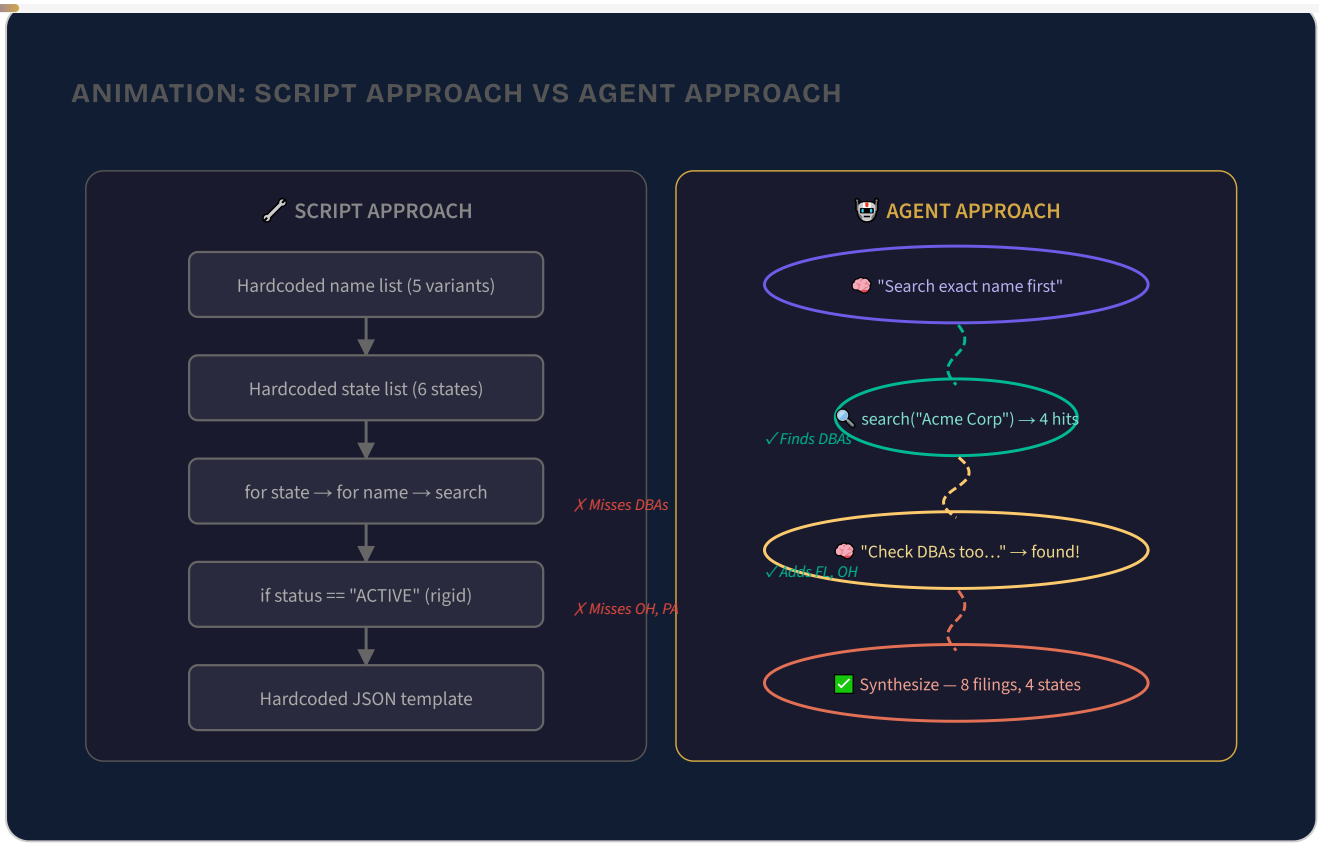
"Agents must be complex — hundreds of lines of code?" — The core agent pattern is about 10 lines: a while loop that calls an LLM, checks if it wants to use a tool, runs the tool, and repeats. Everything else (memory, planning, guardrails) is added incrementally. You'll have a working agent by Module 5.

Script vs Agent — Why This Course Exists

Before we show you a full agent in action, let's answer the obvious question: *why not just write a regular Python script?* Consider the task from the previous section — finding total lien exposure for Acme Corporation. Here's how a script and an agent approach the same problem:

The Core Difference

Aspect	Script	Agent
Name variations	Hardcoded list YOU maintain	Discovered by reasoning at runtime
States to search	Fixed list	Dynamic — based on findings
Decision logic	if/else chains YOU write	Claude reasons about what to do next
New edge cases	Code change + redeploy	Handled by reasoning — no code change
Follow-up questions	Build a new function	Natural conversation continuation
Development time	Days (handle every case)	Hours (define tools + loop)



The Key Insight

> **Why It Matters**
>
> **An agent is a script that replaced the hardcoded decision logic with an LLM.** Instead of YOU writing every if/else and every loop condition, Claude *reasons* about what to do next. Your code provides the **tools** — what the agent *can* do. Claude provides the **logic** — what the agent *should* do. The tool-use loop you'll write (about 15 lines) replaces hundreds of lines of decision logic. But you still write the tools — the agent doesn't magically connect to databases or APIs. You provide the hands. Claude provides the brain.

When Scripts Are Better

Agents are **not** always the answer. A traditional script wins when:

- **The task is 100% deterministic** — no reasoning needed (file format conversion, CSV → JSON)
- **Speed matters more than flexibility** — scripts are 100× faster than an LLM round-trip

- **Cost matters** — every agent turn is an API call, and API calls cost money
- **The logic never changes** — agents add unnecessary complexity to fixed workflows
- **You need guaranteed reproducibility** — same input must produce the exact same output every time

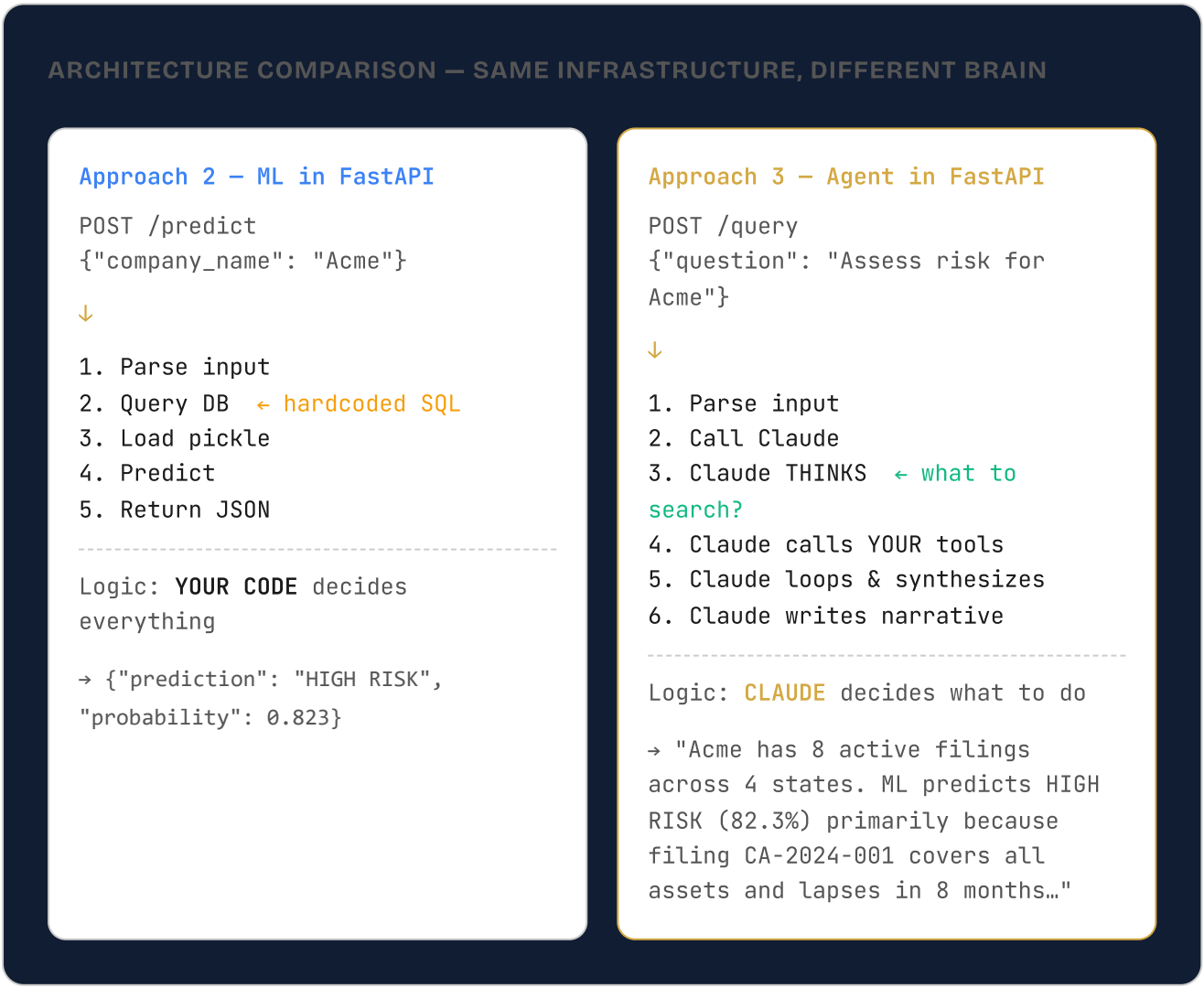
Batch record insertion and cron jobs with fixed logic are script territory. So is data validation against a known schema and simple CRUD operations. This course teaches you to reach for agents when the problem demands reasoning, not as a default for everything.

Why Agents: The Business Case

You just saw three approaches solve the same UCC problem. A sharp reader will notice something: in Approach 2 the ML model is wrapped in FastAPI — and in production the agent (Approach 3) will *also* be wrapped in FastAPI (M22B teaches exactly that). So what is actually different? This section answers that question, then lays out the seven concrete benefits an agent provides, and finally tells you when *not* to use one.

Wait — Both End Up in FastAPI. So What's the Difference?

The infrastructure is identical. What changes is **where the decision logic lives**. Compare the two server bodies side by side:

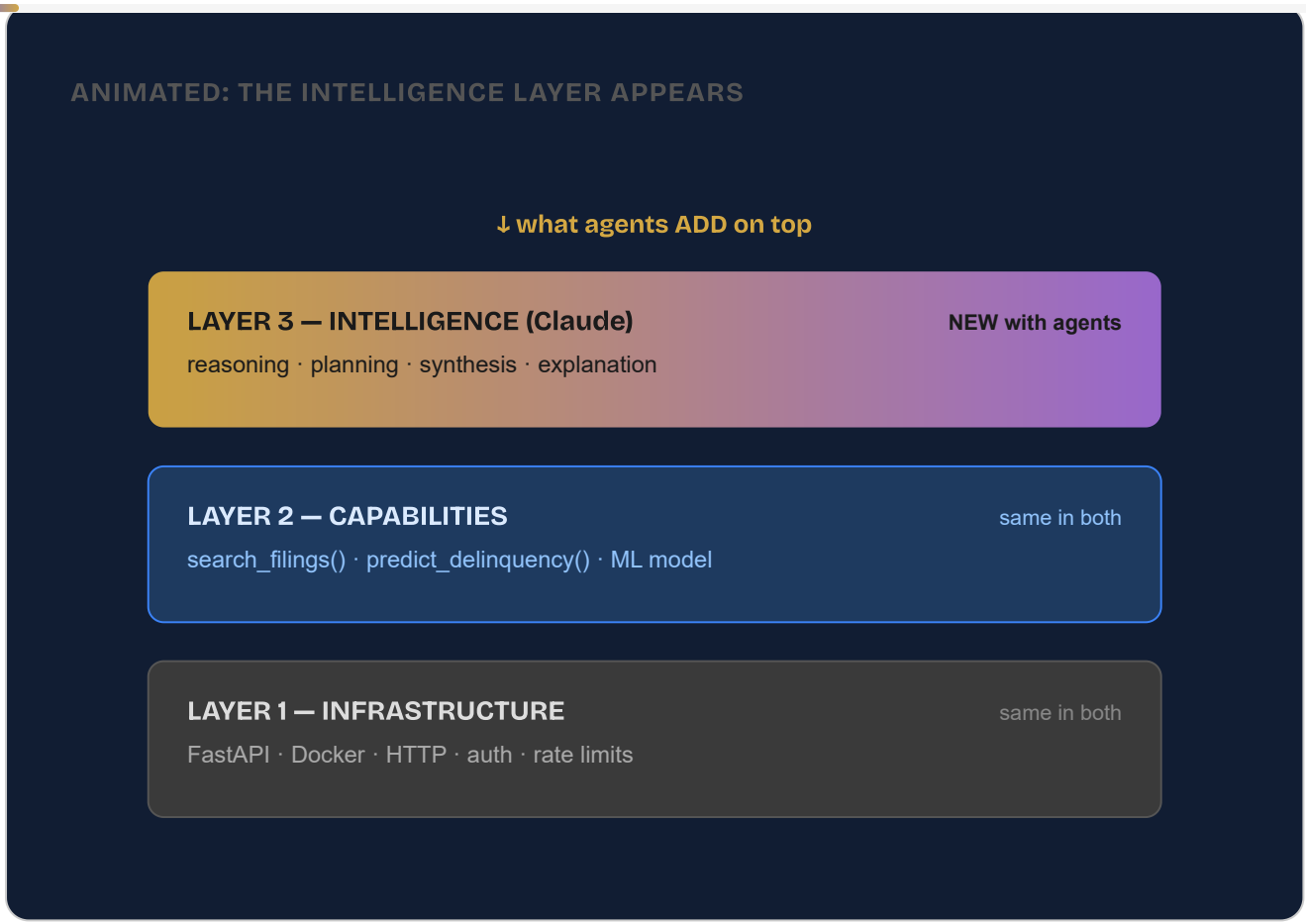


What's the same: FastAPI server, an HTTP endpoint, Docker container, auth, rate limits, deployment to Cloud Run or Lambda. **What's different:** who decides what to do once a request arrives.

Aspect	ML Model in FastAPI	Agent in FastAPI
Who decides what to query?	Your hardcoded SQL	Claude reasons about what to search
Who handles name variations?	Your `ILIKE` pattern	Claude discovers them
Who picks which data to look at?	Your code, fixed order	Claude, based on what it finds
Who formats the response?	Your template	Claude writes natural language
What changes when logic changes?	Your code + redeploy	The system prompt (no redeploy)

The Three-Layer Stack — What Agents Actually Add

The cleanest way to see the difference is as three layers. Two of them are identical in both approaches. The third is what an agent adds.



Layer 1 (Infrastructure) and **Layer 2 (Capabilities, including the ML model)** are identical in both approaches. The ML model *does not move* — it stays in Layer 2 in both. The difference is what sits *above* it: in Approach 2 your hand-written if/else logic; in Approach 3 Claude's reasoning. **Agents don't replace ML models — they put a reasoning layer on top of them.**

> **The Career Insight**
>
> In five years most APIs will still be FastAPI (or equivalent). Most ML models will still be pickle, ONNX, or TensorFlow files. The change is Layer 3 — the reasoning that decides how to use the tools and models. That is what this course teaches you to build.

The Cost-Benefit Reality

An agent costs more per request and is slower. It also collapses development time and produces output a non-technical user can read.

Metric	ML in FastAPI	Agent in FastAPI
Response time	50–200 ms	3–15 seconds
Cost per request	~$0	$0.003–0.075
Development time for v1	2–3 days	4–6 hours
Time to add a new question type	1–2 days (new endpoint)	0 (Claude handles it)
Time to handle a new edge case	Hours (find, code, test, deploy)	0 (Claude reasons about it)
Maintenance burden	High (every change = code)	Low (tools rarely change)
Explainability	Manual feature importance	Built-in narrative
User training	API documentation	None — natural language

The 7 Benefits — UCC Examples

1. Reasoning replaces hardcoded logic. The Approach 2 script searched five hardcoded name variants. The agent discovered `ACME CORP DBA ROADRUNNER SUPPLIES` by reasoning *"let me also check for DBAs"* — a step nobody programmed. A real credit-risk team replaced 200+ name-variation rules with an agent and saw match rate jump from 78% to 94%.

2. Natural language in, structured action out. Traditional: `POST /predict {"company_name": "ACME CORPORATION", "state": "NY"}`. Agent: *"What's the lien exposure for Acme across the northeast?"* Adoption widens from five engineers who know the API to fifty analysts who can just ask.

3. Explainability is built in. ML model says `0.823`. Agent says *"HIGH (82.3%) primarily because filing CA-2024-001 covers all assets and lapses in 8 months — if it lapses without renewal, $2.4M of collateral becomes unsecured."* OCC and FDIC examiners can read the second; they cannot read the first.

4. Follow-up questions without new code. Traditional path requires `/predict`, then `/predict-by-state`, then `/compare-entities`, then `/what-if-continuation` — four

endpoints. The agent handles all four naturally with the same three tools. New question types ship at zero engineering cost.

5. Multi-source synthesis. The prelude agent searched filings (Tool 1), ran the ML model (Tool 2), drilled into the riskiest filing (Tool 3), and wove the three into one report. Writing that synthesis as a script is 200+ lines and breaks when a fourth data source arrives. Claude handles the joining.

6. Graceful handling of incomplete data. Search returns nothing for Texas. A script silently omits TX. The agent says: *"Searched TX — no active filings found. Last filing TX-2021-005 was terminated in 2022."* Analysts stop asking "did you check Texas?" because the agent volunteers what it checked.

7. The ML model gets smarter context (not replaced). The model still sees six numbers. The agent wraps those numbers with which filings produced them, what the collateral descriptions say, and what a continuation or termination would change. The model gives the probability; the agent gives the story. **Agents don't replace ML models — they make them more useful.**

When NOT to Use Agents

Agents are not the right answer for every problem. Reach for a script, an API, or a rule engine when:

Situation	Why Not an Agent
Batch processing 1M records	Agent cost: $10K+. Script cost: ~$0.
Sub-100ms response required	Agent latency: 3–15 s. Script: milliseconds.
Deterministic compliance check	Must be reproducible bit-for-bit. Agents are non-deterministic.
Simple CRUD operations	No reasoning needed. Over-engineering.
No human will read the output	If the next consumer is another system, a structured API is better.

The Decision Rule

If a **human** needs to UNDERSTAND the output, consider an agent. If a **machine** needs to CONSUME the output, use an API. The agent costs $0.015 and 10 extra seconds — cheap if the alternative is a human analyst spending 30 minutes at $50–100/hour, expensive if the next caller is a downstream batch job that just needs a number.

See an Agent in Action — Live Demo Walkthrough

Let's make this concrete. Imagine you work at a bank, and your job is to assess lien risk for corporate borrowers. Today, a loan officer asks: *"What's the total lien exposure for Acme Corporation across all states?"*

Below is exactly what happens behind the scenes when an agent handles this question. This is the same UCC Filing Research Agent that we'll use as a running example throughout the course. Every step maps to concepts you'll learn in specific modules.

The 9-Step Agent Interaction

ANIMATION: UCC FILING RESEARCH AGENT — BEHIND THE SCENES

What Just Happened?

The agent made **three key decisions** that a chatbot never could: (1) it decided it needed to search a database (tool selection), (2) it decided to search again with a name variation (autonomous reasoning), and (3) it decided to pull a risk profile for a complete answer (multi-step planning). Each decision came from Claude's LLM reasoning — the same reasoning you'll learn to harness starting in Module 1.

This walkthrough isn't hypothetical — it's the exact architecture you'll build. By Capstone 1 (after Module 7), you'll have a working version of this agent. By Capstone 5 (after Module 22), you'll have a production-grade version with guardrails, monitoring, and deployment.

The Agent Architecture — Building Blocks Map

Every production agent, no matter how simple or complex, is built from the same 7 components. Think of them like the parts of a human body — each has a specific job, and they all work together. The animation below shows each component lighting up and connecting to the course modules where you'll learn to build it.

Everyday Analogy

Before: Think about a doctor seeing a patient. A medical student who only learned anatomy but never practiced diagnosis would be dangerous. A doctor who can diagnose but has no access to lab tests can't confirm anything. A doctor who can diagnose and order tests but never checks the results is negligent.

The pain: A real doctor needs ALL the skills working together — knowledge (brain), tools (lab access), memory (patient history), judgment (diagnosis plan), safety checks (drug interaction alerts), monitoring (follow-up), and practice setting (clinic or hospital).

The mapping: An AI agent works the same way. The LLM is the brain (knowledge + reasoning). Tools are the hands (database access, API calls). Memory is the patient chart (conversation history, RAG). Planning is the diagnosis process (breaking complex problems into steps). Guardrails are safety protocols (preventing harmful actions). Observability is the monitoring system (logs, traces). Deployment is the practice setting (where it runs). Remove any one piece, and the agent is incomplete.

What this looks like in practice: Here's a simplified agent config showing all 7 blocks wired together: `{"brain": "claude-sonnet-4-20250514", "tools": ["search_filings", "get_risk_profile"], "memory": "conversation_history[]", "plan": "ReAct loop", "guardrails": {"max_turns": 10, "banned_tools": ["delete_filing"]}, "observability": "langfuse_trace_id", "deployment": "cloud_run_endpoint"}`. Each field maps to a building block. By Module 22, you'll understand every one of these.

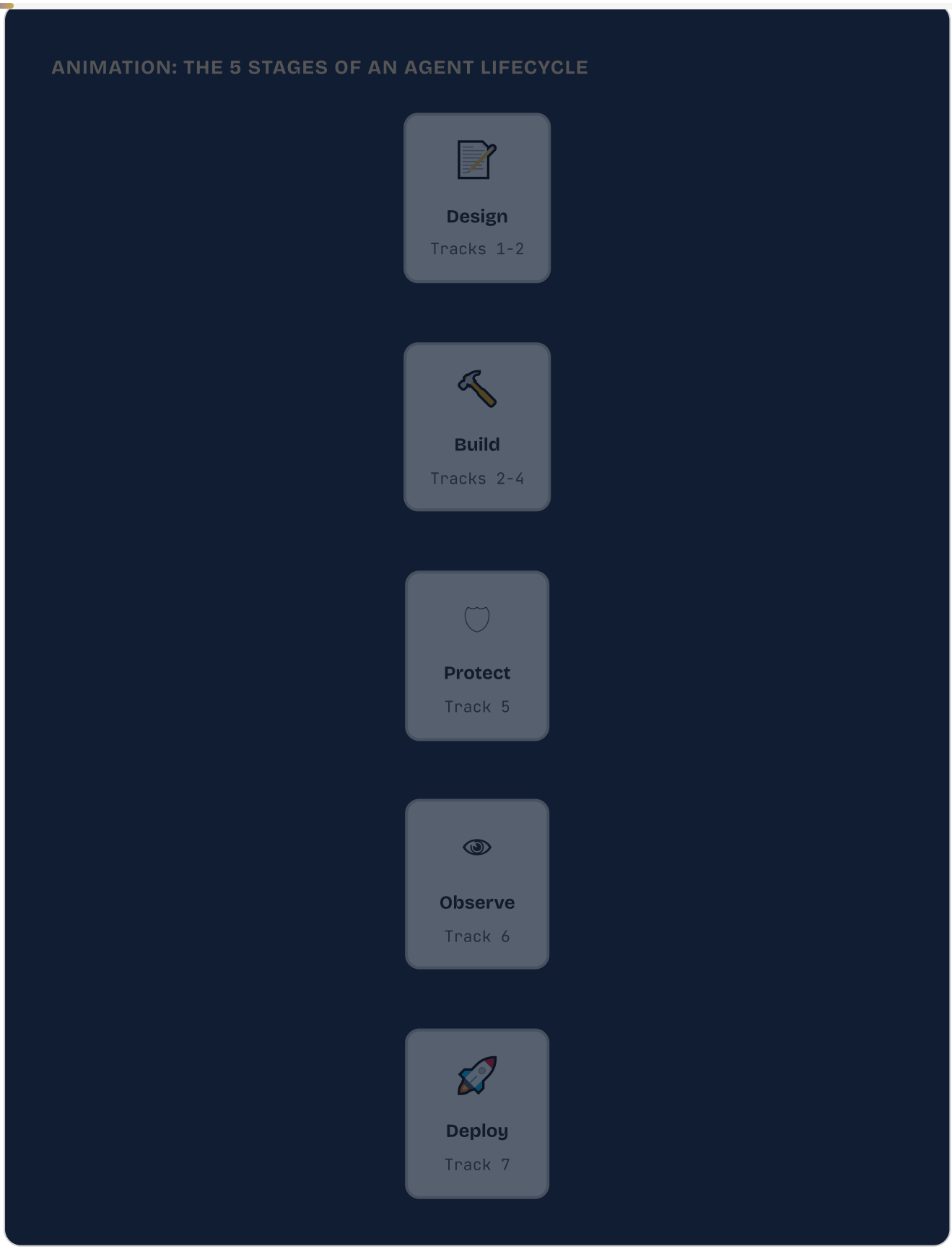
Here's what each component does and why it matters:

Component	What It Does	Without It...	Course Track
Brain (LLM)	Reads input, reasons about it, decides what to do next, generates text	No reasoning — just static rules or pattern matching	Track 1: M01-M04
Tools	Calls APIs, queries databases, reads files, sends emails	Can only respond from training data — no real-time info	Track 2: M05-M07
Memory	Remembers conversation history, retrieves relevant documents (RAG)	Forgets everything between messages — asks "who are you?" every turn	Track 3: M08-M11
Plan	Breaks complex tasks into subtasks, decides execution order	Can only handle simple, one-step requests	Track 4: M12-M15
Guardrails	Validates inputs, checks outputs, escalates to humans when unsure	No safety net — generates harmful content, runs expensive loops	Track 5: M16-M18
Eyes	Logs every decision, traces tool calls, monitors performance	A black box — when it breaks, you have no idea why	Track 6: M19-M20
Home	Runs in production: API design, containerization, scaling, cost control	Works on your laptop, but nobody else can use it	Track 7: M21-M22

"Most tutorials stop at the Brain and Tools — that's only 2 of 7 components. This course covers all seven, because an agent that works on your laptop but can't be trusted in production isn't useful."

The Agent Lifecycle — From Idea to Production

Building an agent isn't just "write code and ship it." Production agents go through 5 stages, each adding a critical layer. The animation below shows these stages flowing left-to-right, with the course tracks mapped to each.



Let's walk through each stage with the UCC Filing Research Agent as our example:

1. **Design** — What should the agent do? What tools does it need? What data does it access? For our UCC agent: it needs to search a filing database, handle name variations, and produce risk assessments. You'll learn how LLMs think (M01-M04) and how to define tools (M05-M07) in this stage.
2. **Build** — Write the code. This means defining tools, engineering prompts, and wiring up the agent loop. You might also add a RAG pipeline for policy documents. This is where the UCC agent gets its `search_filings` and `get_risk_profile` tools wired up. Covered in Tracks 2-4 (M05-M15).
3. **Protect** — What if someone asks the agent to delete filings? What if it hallucinates a risk score? What if it runs 500 tool calls and costs $100? This stage adds guardrails to validate inputs and outputs. It also adds human-in-the-loop approval for high-stakes actions and cost controls to prevent runaway spending. Track 5 (M16-M18).
4. **Observe** — When the agent produces a wrong answer, how do you figure out why? Tracing every LLM decision, logging tool calls, monitoring accuracy over time. Without this, production agents are black boxes. Track 6 (M19-M20).
5. **Deploy** — Ship it: wrap the agent in an API, containerize it, deploy to cloud, optimize costs. The UCC agent becomes a service that any analyst in the bank can call. Track 7 (M21-M22).

> **Most Tutorials Stop at Stage 2**
>
> Search "build an AI agent" on YouTube and you'll find hundreds of tutorials that cover Design and Build. Almost none cover Protect, Observe, or Deploy. This course covers **all five stages** — because an agent that works in a demo but breaks in production, costs too much to run, or can't be debugged when it fails isn't a real product. It's a prototype.

Common Misconceptions About the Agent Lifecycle

"I can add guardrails later, after the agent works." — Dangerous approach. By the time you discover your agent can hallucinate risk scores or run 200 tool calls per query, you've already deployed it. Build guardrails into your agent loop from the start, even if they're simple (like a max iteration count).

"Observability means just adding print statements." — Print statements disappear when the server restarts. Production observability means structured logs, distributed traces, and dashboards that show you *why* the agent chose a particular tool on a specific request last Tuesday. You'll learn this in Modules 19-20.

"The lifecycle is waterfall — finish one stage before starting the next." — Not at all. You'll iterate between stages constantly. Building a new tool? You also need a guardrail for it and a trace for it. The lifecycle describes what concerns exist, not a strict order of operations.

How Agents Actually Work — The API Reality

Here's a common misconception: many people think an AI agent is a separate application running somewhere, making decisions on its own. The reality is much simpler — and more empowering, because it means *you* are always in control.

An agent is **your code** that calls Claude's API in a loop. There's no magic runtime, no background daemon, no persistent process making autonomous choices. Your code sends a request. Claude's servers send a response. Your code decides what to do with that response. Repeat.

This matters because it means the agent only runs when your code tells it to. Between API calls, nothing is happening. The agent doesn't "think" while waiting. It doesn't accumulate knowledge between sessions (unless you explicitly save state). Every call is stateless — you send the full conversation history each time, and Claude responds as if it's seeing the conversation for the first time.

Here's the entire pattern in pseudocode — every agent, from a simple calculator to a multi-million-dollar enterprise system, is a variation of this:

```
// The universal agent pattern
user asks a question

while (claude wants to use a tool):
    send the question (+ history) to claude
    if claude returns a tool request:
        run the tool
        send the result back to claude
    else:
        return claude's answer to the user
```

What Just Happened?

Look at that pseudocode again. The `while` loop is the entire agent. Your code sends a message to Claude, and Claude responds with either a **tool request** ("please call `search_filings` with these inputs") or a **final answer** ("here's your report"). If it's a tool request, you run the tool and send the result back — that's one iteration. If it's a final answer, you return it to the user and the loop ends. That's it. Really.

The rest of this course teaches you every variation, optimization, and production concern around this loop. You'll learn how Claude decides whether to call a tool (Module 5), how to handle multiple tools at once (Module 6), how to give Claude access to your private data (Module 9), how to prevent infinite loops (Module 16), and how to trace every step for debugging (Module 19).

Where Agents Run

During development, your agent runs on your laptop — you type a command, the loop executes, you see the result. This is how you'll work through most of this course.

In production, the same loop runs on a server. Typically it sits behind a REST API or webhook that receives user requests and returns agent responses. A loan officer clicks "Analyze" in a web app, which sends a request to your server, which runs the agent loop, which calls Claude's API, which calls your tools, and returns the final answer. The loan officer never sees the loop — just the result.

Some agents don't have a user at all. They run in CI/CD pipelines (automated code review on every pull request), on cron schedules (daily compliance report generation), or inside chat interfaces like Slack (customer support bots). The loop is the same everywhere — only the trigger changes. You'll learn to deploy agents in all these contexts in Modules 21-22.

Three Agents You'll Build in This Course

Theory only goes so far. This course has 5 capstone projects that put everything together. Here are three of them, at increasing difficulty, to show you what you're working toward:

Capstone 1: Filing Lookup Agent

A simple agent that answers questions about UCC filings using a database tool. One tool, one loop, immediate results.

`Difficulty: ★☆☆☆☆ · ~30 min · After M07`

Capstone 3: Research Agent

A multi-tool research agent that reasons through complex questions, calling different tools based on what it discovers, handling ambiguity and name variations.

`Difficulty: ★★★☆☆ · ~90 min · After M15`

Capstone 5: Production System

A full production agent pipeline with planning, memory, guardrails, human oversight, model routing, evaluation suite, and deployment. The real deal.

`Difficulty: ★★★★★ · ~4-6 hours · After M22`

You start at Capstone 1 in Module 7. By Module 22, you'll build Capstone 5. The jump from one to five isn't a leap — it's a staircase, and every module is one step.

How a Claude Agent Built This Course

Before we start teaching you to build agents, let us show you one in action — **the one that built the course you're reading right now**.

This entire course — every module, every animation, every quiz question, every code example — was generated by an AI agent using the exact same patterns you're about to learn. The agent is called Claude Code, and it's Anthropic's own agentic coding tool.

Claude Code is an AI agent that reads files, writes code, executes terminal commands, and iterates until the output meets quality standards. It runs locally on the course author's machine. It uses Claude (the same model you'll use) as its LLM brain. And it follows the *exact same architecture* that you'll build in this course.

The Course-Building Agent Architecture

Here's what the course-building agent looks like. Every component maps to a concept you'll learn:

ANIMATION: THE AGENT THAT BUILT THIS COURSE

Let's map each component to the course modules where you'll learn to build it yourself:

Agent Component	What It Did for This Course	Where You'll Learn It
`CLAUDE.md`	Project memory — told the agent the design system, quality standards, and depth rules to follow	M25 (Claude Code Mastery)
Slash commands	Predefined workflows: `/generate-module`, `/fix-explanations`, `/review-module`	M25 (Claude Code Mastery)
Prompt files	Loaded design specs, module briefs, and cert tips on-demand — like RAG without a vector database	M09 (RAG), M25 (Claude Code)
Read / Write / Edit tools	Read existing modules for consistency, wrote new HTML files, edited specific sections	M05 (Function Calling)
Bash tool	Ran file checks, counted sections, verified HTML structure	M15 (Code Interpreter)
ReAct loop	Think → Read → Generate → Review → Edit → Repeat until quality passes	M12 (ReAct Pattern)
Quality checklist	16-point validation after every module (guardrails for the agent)	M16-M17 (Guardrails)

But How Does Claude Actually Build the HTML?

This is the question students always ask — and the answer reveals something important about how LLMs work.

Claude doesn't use a website builder, a template engine, or a React framework. There's no WordPress, no static site generator, no drag-and-drop editor. Here's what *actually* happens:

Step 1: Load context (the Read tool). The agent reads specification files from disk — the CSS design system, the 14 depth rules for explanation quality, the module brief describing what concepts to cover. These files go into Claude's context window — its working memory for this task. This is functionally identical to the RAG pattern you'll learn in Module 9: load relevant knowledge before generating.

Step 2: Generate HTML (the LLM brain). Claude learned HTML, CSS, and JavaScript during training — from millions of web pages, documentation sites, and interactive tutorials. When it generates a module, it combines the design specifications (from Step 1) with its knowledge of web technologies to produce a complete, self-contained HTML file. It writes this character by character, token by token — just like it writes any text response. There is no template with blanks to fill.

Step 3: Write the file (the Write tool). The agent saves the generated HTML to disk — about 100-200KB of self-contained code with all CSS and JavaScript inline.

Step 4: Review its own work (Read + Grep tools). The agent reads the file back and checks it against quality rules. Does it have at least 5 quiz questions? Are all sections present? Are there ARIA labels for accessibility? This is the agent's guardrail — reviewing its own output before reporting success.

Step 5: Edit specific sections (the Edit tool). When the human says "make the animation slower," Claude doesn't regenerate the entire 150KB file. It finds the specific CSS value and changes it. Only that line changes. This makes iterations fast (seconds) and precise (no unintended side effects).

The Workflow — Step by Step

The Key Insight

The course-building agent uses **no special capabilities**. It uses the same tools and patterns you'll learn: file reading (M05), file writing (M05), code execution (M15), loading specs before generating (RAG, M09), generate → review → fix loop (ReAct, M12), and quality checks (Guardrails, M16-M17). The "magic" isn't in the tools — it's in the **specifications** (prompt files that define exactly what to generate) and the **iteration loop** (review and fix until quality passes). By Module 22B, you'll be able to build a system like this yourself.

The Numbers

Metric	Value
Time per module (agent work)	~4 minutes
Time per module (human review)	~10-15 minutes
Total modules generated	30 (including capstones)
Total API cost	~$20-30
Output per module	80-200KB self-contained HTML

Course Roadmap — What You'll Learn and When

Here's the complete course map. Nine tracks, 28 modules (plus 2 BUILD modules), and 5 capstone projects. Each track builds on the previous one.

Choose Your Learning Path

Not everyone needs every module. Here are three paths depending on your goal:

Path A — Weekend Builder

Build and run a working agent in one weekend. Skip the theory you can learn later.

```
M01 → M03 → M05 → M12 → M15B → Capstone 1
~6-8 hours total
```

Path B — Deep Diver

Go through every module in order. The most comprehensive understanding of agent development.

```
M00 → M27 sequentially (including M15B and M22B)
~60-80 hours total
```

Path C — Cert Prep

Prepare for the Claude Certified Architect exam. Focused on the 5 exam domains.

```
M01 → M24 → M25 → M26 → M27
~40-50 hours total
```

Choose your path, or take all three. Either way, start with M01 after this module — it's where the real learning begins.

Reflection Exercise

This module is intentionally code-free. Before you start building in M01, take a moment to think about *your* agent ideas.

Reflection 1: Your Agent Idea

Think of a repetitive task in your work that involves: **looking up information**, **making decisions based on rules**, and **producing a report or response**. That task is a candidate for an agent. Write it down — by the end of this course, you'll be able to build it.

Examples: reviewing insurance claims against policy criteria, checking inventory levels and reordering, answering customer questions from a knowledge base, analyzing financial filings for risk factors.

Reflection 2: Generate → Review → Fix

The course-building agent follows a simple pattern: generate output, review it against quality rules, fix the issues, repeat. What's a similar **generate → review → fix** workflow in your domain that an agent could handle?

Examples: drafting legal documents and checking them against templates, generating test cases and verifying they cover edge cases, writing marketing copy and ensuring brand guideline compliance.

Knowledge Check

Test your understanding of the agent concepts introduced in this module. You need to understand these before moving to M01.

Q1: What's the key difference between a chatbot and an agent?

- ○ A chatbot uses a better LLM model than an agent
- ○ An agent runs autonomously 24/7 without human input
- ○ An agent can use tools, make decisions, and loop until the task is complete
- ○ A chatbot can only respond in text, while an agent can respond with images

Q2: In the UCC Filing Research Agent demo, why did the agent search the database twice?

- ○ The first search returned an error
- ○ It decided to check for name variations (e.g., "ACME CORP" vs "Acme Corporation")
- ○ The database required two queries for pagination
- ○ It was programmed to always search exactly twice

Q3: Which building block provides the agent's "hands" — the ability to interact with the outside world?

- ○ Tools (function calling, MCP servers)
- ○ Memory (conversation history, RAG)
- ○ The Brain (LLM)
- ○ Guardrails (input/output validation)

Q4: A production agent needs more than just working code. Which TWO additional requirements does the agent lifecycle include?

- ○ A custom UI and a mobile app
- ○ Machine learning training and fine-tuning
- ○ A blockchain ledger and distributed consensus
- ○ Guardrails (safety controls) and Observability (monitoring/tracing)

Q5: What is the basic pattern of every agent?

- ○ Train a model, deploy it, let it respond to queries
- ○ Send a message to the LLM, return whatever it says

- A loop that calls the LLM, checks if a tool is needed, runs the tool, and repeats until done
- A series of if/else rules that decide which pre-written response to return

Q6: What role did CLAUDE.md play in the course-building agent?

- It contained the HTML templates that Claude filled in
- It served as project memory — rules, standards, and design system that persisted across sessions
- It stored the conversation history between the author and Claude
- It was the agent's source code

Q7: The course-building agent loaded prompt files before generating each module. Which course concept is this most similar to?

- RAG (Retrieval-Augmented Generation) — loading relevant context before generating
- Fine-tuning — retraining the model on new data
- Guardrails — validating output against rules

- [ ] Multi-agent orchestration — distributing work across agents

Module Summary

You've seen the whole movie. Here's what you now know:

- **An agent = LLM + Tools + Loop.** It doesn't just talk — it thinks, acts, observes, and repeats until the job is done.
- **7 building blocks** make up a production agent: Brain, Tools, Memory, Plan, Guardrails, Eyes (observability), and Home (deployment).
- **5 lifecycle stages**: Design → Build → Protect → Observe → Deploy. Most tutorials stop at Build.
- **The universal agent pattern** is a while loop: call LLM → check for tool use → run tool → repeat.
- **This course was built by an agent** using the same patterns you'll learn — Read → Generate → Review → Edit → Repeat.

> **What's Next: M01 — The LLM Mental Model**
>
> Now you understand the big picture. In Module 1, you'll zoom in on the Brain — the LLM that powers every agent. You'll learn how Claude actually generates text (it's not looking up answers in a database), why "temperature" changes the output, and what tokens are. This is the foundation everything else builds on.